



# CSE 537 - Artificial Intelligence: Project 01 Report

## The Searchin' Pac-Man

**Course:** CSE 537 - Artificial Intelligence

**Assignment:** Project 01 - Search Algorithms

**Language:** Python 3.11

---

**AI Declaration:** Google Antigravity CLI and Chatgpt has been utilized to beautify, structure and formulate scientific notations (ex:  $f(n) = g(n) + h(n)$ ..) the below report under proper supervision and instructions.

---

## 1. Executive Summary

This report presents the implementation, empirical results, and critical analysis for classical graph search algorithms applied to Pac-Man maze navigation and the 8-puzzle problem. The project covers uninformed search strategies (Depth-First Search, Breadth-First Search, Uniform-Cost Search) and informed search strategies (A\* Search) with problem-specific admissible heuristics.

### Key Achievements:

- **Depth-First Search (Q1):** Implemented complete Graph-Search DFS using `util.Stack`.
- **Breadth-First Search (Q2):** Implemented complete Graph-Search BFS using `util.Queue`, verified on Pac-Man mazes and the 8-puzzle.
- **Uniform-Cost Search (Q3):** Implemented UCS using `util.PriorityQueue`, verified across uniform, exponentially decaying ( $0.5^x$ ), and exponentially growing ( $2^x$ ) cost functions.
- **A Search (Q4)\*:** Implemented A\* Search using  $f(n) = g(n) + h(n)$ , demonstrating

superior node reduction over UCS/BFS on `bigMaze` and `openMaze` .

- **Corners Problem Formulation (Q5):** Designed a compact abstract state representation `((x, y), (c0_visited, c1_visited, c2_visited, c3_visited))` for visiting all 4 maze corners.
- **Corners Problem Heuristic (Q6):** Formulated an admissible permutation-based minimum Manhattan path heuristic, achieving **741** expanded nodes on `mediumCorners` (placing in the top performance tier  $< 800$  nodes).
- **Food Search Heuristic (Q7):** Formulated an admissible and consistent cached maximum maze-distance heuristic for eating all food dots, achieving **4,137** expanded nodes in **0.3 seconds** on `trickySearch` .

---

## 2. Master Performance & Empirical Statistics Table

| Q# | Search Algorithm | Problem / Layout            | Cost Function / Heuristic | Solution Cost | Nodes Expanded |
|----|------------------|-----------------------------|---------------------------|---------------|----------------|
| Q1 | DFS              | <code>tinyMaze</code>       | Uniform (1)               | 10            | 15             |
| Q1 | DFS              | <code>mediumMaze</code>     | Uniform (1)               | 130           | 146            |
| Q1 | DFS              | <code>bigMaze</code>        | Uniform (1)               | 210           | 390            |
| Q2 | BFS              | <code>mediumMaze</code>     | Uniform (1)               | <b>68</b>     | 269            |
| Q2 | BFS              | <code>bigMaze</code>        | Uniform (1)               | <b>210</b>    | 620            |
| Q2 | BFS              | <code>eightpuzzle.py</code> | Uniform (1)               | 1             | N/A            |

| Q# | Search Algorithm | Problem / Layout | Cost Function / Heuristic       | Solution Cost  | Nodes Expanded |
|----|------------------|------------------|---------------------------------|----------------|----------------|
|    |                  |                  |                                 |                |                |
| Q3 | UCS              | mediumMaze       | SearchAgent (Uniform)           | 68             | 269            |
| Q3 | UCS              | mediumDottedMaze | StayEastSearchAgent ( $0.5^x$ ) | 1              | 186            |
| Q3 | UCS              | mediumScaryMaze  | StayWestSearchAgent ( $2^x$ )   | 68,719,479,864 | 108            |
| Q4 | A*               | bigMaze          | manhattanHeuristic              | 210            | 549            |
| Q4 | DFS              | openMaze         | Uniform (1)                     | 298            | 576            |
| Q4 | BFS / UCS        | openMaze         | Uniform (1)                     | 54             | 682            |
| Q4 | A*               | openMaze         | manhattanHeuristic              | 54             | 535            |
| Q5 | BFS              | tinyCorners      | CornersProblem                  | 28             | 252            |

| Q# | Search Algorithm | Problem / Layout | Cost Function / Heuristic | Solution Cost | Nodes Expanded |
|----|------------------|------------------|---------------------------|---------------|----------------|
| Q5 | BFS              | mediumCorners    | CornersProblem            | 106           | 1,966          |
| Q6 | A*               | mediumCorners    | cornersHeuristic          | 106           | 741            |
| Q7 | A*               | testSearch       | foodHeuristic             | 7             | 10             |
| Q7 | A*               | trickySearch     | foodHeuristic             | 60            | 4,137          |

---

### 3. Question-by-Question Detailed Analysis & Source Code

---

# Question 1: Depth-First Search (DFS)

## 3.1.1 Implementation ( search.py )

```
def depthFirstSearch(problem: SearchProblem) -> List[Directions]:
    """
    Search the deepest nodes in the search tree first.
    """
    fringe = util.Stack()
    start_state = problem.getStartState()
    fringe.push((start_state, []))
    visited = set()

    while not fringe.isEmpty():
        state, actions = fringe.pop()

        if problem.isGoalState(state):
            return actions

        if state not in visited:
            visited.add(state)
            for successor, action, stepCost in problem.getSuccessors(state):
                if successor not in visited:
                    fringe.push((successor, actions + [action]))

    return []
```

## 3.1.2 Empirical Statistics & Execution Table

| Layout     | Command Executed                                 | Solution Path Cost | Nodes Expanded |
|------------|--------------------------------------------------|--------------------|----------------|
| tinyMaze   | python pacman.py -l tinyMaze -p SearchAgent      | 10                 | 15             |
| mediumMaze | python pacman.py -l mediumMaze -p SearchAgent    | 130                | 146            |
| bigMaze    | python pacman.py -l bigMaze -z .5 -p SearchAgent | 210                | 390            |

### 3.1.3 Observations & Conceptual Answers

1. **Exploration Order:** DFS pops states from the LIFO stack in reverse order of pushing. For successor order [North, South, East, West], West is popped and explored first.
  2. **Physical Traversal vs. Search Nodes:** Pac-Man does **not** physically travel to all expanded red squares. Red squares represent state-space exploration; Pac-Man physically travels only along the final returned action list.
  3. **Optimality:** DFS is **not optimal**. On `mediumMaze`, DFS returns a path of cost **130** (expanding 146 nodes), whereas the optimal path length is **68**.
- 

## Question 2: Breadth-First Search (BFS)

### 3.2.1 Implementation ( `search.py` )

```
def breadthFirstSearch(problem: SearchProblem) -> List[Directions]:
    """Search the shallowest nodes in the search tree first."""
    fringe = util.Queue()
    start_state = problem.getStartState()
    fringe.push((start_state, []))
    visited = set()

    while not fringe.isEmpty():
        state, actions = fringe.pop()

        if problem.isGoalState(state):
            return actions

        if state not in visited:
            visited.add(state)
            for successor, action, stepCost in problem.getSuccessors(state):
                if successor not in visited:
                    fringe.push((successor, actions + [action]))

    return []
```

### 3.2.2 Empirical Statistics & Execution Table

| Layout / Problem | Command Executed                                                        | Solution Path Cost |
|------------------|-------------------------------------------------------------------------|--------------------|
| mediumMaze       | <code>python pacman.py -l mediumMaze -p SearchAgent -a fn=bfs</code>    | <b>68</b>          |
| bigMaze          | <code>python pacman.py -l bigMaze -p SearchAgent -a fn=bfs -z .5</code> | <b>210</b>         |
| eightpuzzle.py   | <code>python eightpuzzle.py</code>                                      | <b>1</b>           |

### 3.2.3 Observations & Conceptual Answers

1. **Optimality Proof:** In unweighted graphs (all step costs  $c = 1$ ), path cost equals path depth. Because BFS expands nodes level-by-level, the first goal state dequeued is guaranteed to have the minimum possible path cost (**68** on `mediumMaze` vs **130** for DFS).
-

## Question 3: Uniform-Cost Search (UCS) & Varying Cost Functions

### 3.3.1 Implementation ( search.py )

```
def uniformCostSearch(problem: SearchProblem) -> List[Directions]:
    """Search the node of least total cost first."""
    fringe = util.PriorityQueue()
    start_state = problem.getStartState()
    fringe.push((start_state, [], 0), 0)
    visited = set()

    while not fringe.isEmpty():
        state, actions, cost = fringe.pop()

        if problem.isGoalState(state):
            return actions

        if state not in visited:
            visited.add(state)
            for successor, action, stepCost in problem.getSuccessors(state):
                if successor not in visited:
                    new_cost = cost + stepCost
                    fringe.push((successor, actions + [action], new_cost), new_cost)

    return []
```

### 3.3.2 Empirical Statistics & Execution Table

| Layout           | Agent / Cost Function              | Command Executed                                            |
|------------------|------------------------------------|-------------------------------------------------------------|
| mediumMaze       | SearchAgent<br>(Uniform)           | python pacman.py -l mediumMaze -p SearchAgent               |
| mediumDottedMaze | StayEastSearchAgent<br>( $0.5^x$ ) | python pacman.py -l mediumDottedMaze -p StayEastSearchAgent |
| mediumScaryMaze  | StayWestSearchAgent                | python pacman.py -l mediumScaryMaze -p StayWestSearchAgent  |

| Layout | Agent / Cost Function | Command Executed |
|--------|-----------------------|------------------|
|        | $(2^x)$               |                  |

### 3.3.3 Mathematical Analysis of Cost Difference

- **SearchAgent (mediumMaze)**: Uniform step cost  $\text{cost}(x, y) = 1$ . Total cost =  $68 \times 1 = 68$ .
  - **StayEastSearchAgent (mediumDottedMaze)**: Decaying cost  $\text{cost}(x, y) = 0.5^x = \frac{1}{2^x}$ . As Pac-Man moves further East (increasing  $x$ ), step costs drop exponentially (0.5, 0.25, 0.125, ...). Summing these tiny fractional step costs yields a total cost of **1.0**.
  - **StayWestSearchAgent (mediumScaryMaze)**: Exponentially growing cost  $\text{cost}(x, y) = 2^x$ . Stepping into positions with large  $x$  coordinates carries huge penalties ( $2^{30} = 1,073,741,824$ ;  $2^{36} = 68,719,476,736$ ). Reaching a goal at column  $x = 36$  incurs a total accumulated path cost of **68,719,479,864**.
-

## Question 4: A\* Search & openMaze Comparison

### 3.4.1 Implementation ( search.py )

```
def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic) -> List[Directions]:
    """Search the node that has the lowest combined cost and heuristic first."""
    fringe = util.PriorityQueue()
    start_state = problem.getStartState()
    fringe.push((start_state, [], 0), 0 + heuristic(start_state, problem))
    visited = set()

    while not fringe.isEmpty():
        state, actions, cost = fringe.pop()

        if problem.isGoalState(state):
            return actions

        if state not in visited:
            visited.add(state)
            for successor, action, stepCost in problem.getSuccessors(state):
                if successor not in visited:
                    new_cost = cost + stepCost
                    priority = new_cost + heuristic(successor, problem)
                    fringe.push((successor, actions + [action], new_cost), priority)

    return []
```

### 3.4.2 Empirical Statistics & Execution Table

| Layout   | Search Strategy / Heuristic | Command Executed                                     |
|----------|-----------------------------|------------------------------------------------------|
| bigMaze  | A*<br>(manhattanHeuristic)  | python pacman.py -l bigMaze -z .5 -p SearchAgent -a  |
| openMaze | DFS                         | python pacman.py -l openMaze -p SearchAgent -a fn=df |

| Layout   | Search Strategy / Heuristic | Command Executed                                                  |
|----------|-----------------------------|-------------------------------------------------------------------|
| openMaze | BFS                         | <code>python pacman.py -l openMaze -p SearchAgent -a fn=bf</code> |
| openMaze | UCS                         | <code>python pacman.py -l openMaze -p SearchAgent -a fn=uc</code> |
| openMaze | A*<br>(manhattanHeuristic)  | <code>python pacman.py -l openMaze -p SearchAgent -a fn=as</code> |

### 3.4.3 Search Strategy Comparison on openMaze

- **DFS:** Wanders through open space, returning a highly sub-optimal path of cost **298**.
  - **BFS / UCS:** Expands in concentric circles in all directions (**682** nodes).
  - **A\*:** Uses Manhattan distance to direct search straight toward the goal (**535** nodes).
-

## Question 5: `CornersProblem` State Formulation

### 3.5.1 Implementation ( `searchAgents.py` )

```

class CornersProblem(search.SearchProblem):
    def __init__(self, startingGameState: pacman.GameState):
        self.walls = startingGameState.getWalls()
        self.startingPosition = startingGameState.getPacmanPosition()
        top, right = self.walls.height-2, self.walls.width-2
        self.corners = ((1,1), (1,top), (right, 1), (right, top))
        for corner in self.corners:
            if not startingGameState.hasFood(*corner):
                print('Warning: no food in corner ' + str(corner))
        self._expanded = 0

    def getStartState(self):
        visited = tuple(self.startingPosition == corner for corner in self.corners)
        return (self.startingPosition, visited)

    def isGoalState(self, state: Any):
        position, visited = state
        return all(visited)

    def getSuccessors(self, state: Any):
        successors = []
        position, visited = state
        x, y = position

        for action in [Directions.NORTH, Directions.SOUTH, Directions.EAST, Directions.WEST]:
            dx, dy = Actions.directionToVector(action)
            nextx, nexty = int(x + dx), int(y + dy)
            if not self.walls[nextx][nexty]:
                next_pos = (nextx, nexty)
                new_visited = list(visited)
                for i, corner in enumerate(self.corners):
                    if next_pos == corner:
                        new_visited[i] = True
                successors.append(((next_pos, tuple(new_visited)), action, 1))

        self._expanded += 1
        return successors

```

### 3.5.2 Empirical Statistics & Execution Table

| Layout                     | Problem                     | Command Executed                                                  |
|----------------------------|-----------------------------|-------------------------------------------------------------------|
| <code>tinyCorners</code>   | <code>CornersProblem</code> | <code>python pacman.py -l tinyCorners -p SearchAgent -a fn</code> |
| <code>mediumCorners</code> | <code>CornersProblem</code> | <code>python pacman.py -l mediumCorners -p SearchAgent -a</code>  |

---

## Question 6: `cornersHeuristic`

### 3.6.1 Implementation ( `searchAgents.py` )

```
def cornersHeuristic(state: Any, problem: CornersProblem):
    corners = problem.corners # These are the corner coordinates
    walls = problem.walls # These are the walls of the maze, as a Grid (game.py)

    position, visited = state
    unvisited = [corners[i] for i in range(len(corners)) if not visited[i]]

    if not unvisited:
        return 0

    import itertools
    min_dist = float('inf')
    for perm in itertools.permutations(unvisited):
        dist = util.manhattanDistance(position, perm[0])
        for i in range(len(perm) - 1):
            dist += util.manhattanDistance(perm[i], perm[i+1])
        if dist < min_dist:
            min_dist = dist

    return min_dist
```

### 3.6.2 Empirical Statistics & Execution Table

| Layout                     | Agent / Heuristic                                                   | Command Executed                                               |
|----------------------------|---------------------------------------------------------------------|----------------------------------------------------------------|
| <code>mediumCorners</code> | <code>AStarCornersAgent</code><br>( <code>cornersHeuristic</code> ) | <code>python pacman.py -l mediumCorners -p AStarCorners</code> |

## Question 7: foodHeuristic

### 3.7.1 Implementation ( searchAgents.py )

```
def foodHeuristic(state: Tuple[Tuple, List[List]], problem: FoodSearchProblem):
    position, foodGrid = state
    food_list = foodGrid.asList()

    if not food_list:
        return 0

    if 'distances' not in problem.heuristicInfo:
        problem.heuristicInfo['distances'] = {}

    distances = problem.heuristicInfo['distances']

    max_dist = 0
    for food in food_list:
        key = (position, food)
        if key not in distances:
            distances[key] = mazeDistance(position, food, problem.startingGameState)
        dist = distances[key]
        if dist > max_dist:
            max_dist = dist

    return max_dist
```

### 3.7.2 Empirical Statistics & Execution Table

| Layout       | Agent / Heuristic                         | Command Executed                                         |
|--------------|-------------------------------------------|----------------------------------------------------------|
| testSearch   | AStarFoodSearchAgent<br>( foodHeuristic ) | python pacman.py -l testSearch -p AStarFoodSearchAgent   |
| trickySearch | AStarFoodSearchAgent<br>( foodHeuristic ) | python pacman.py -l trickySearch -p AStarFoodSearchAgent |

| Layout | Agent / Heuristic | Command Executed |
|--------|-------------------|------------------|
|        |                   |                  |

### 3.7.3 Proof of Admissibility and Consistency

- **Admissibility:** Pac-Man must visit all remaining food dots, including the farthest dot in maze distance. Therefore,  $h(s) = \max_{f \in \text{food}} \text{mazeDistance}(pos, f) \leq h^*(s)$  is a strict lower bound.
- **Consistency:** Moving 1 step changes distance to any dot by at most 1, satisfying  $h(s) - h(s') \leq c(s, a, s') = 1$ .

## 4. Conclusion & Summary of Findings

Across all 7 questions, informed search algorithms (A\*) combined with well-designed admissible heuristics dramatically outperform uninformed search methods (DFS, BFS, UCS):

1. On `mediumCorners`, A\* with `cornersHeuristic` reduced node expansion by **62.3%** compared to BFS (**741** vs **1,966** nodes).
2. On `trickySearch`, A\* with `foodHeuristic` reduced node expansion from over 16,000 (UCS) down to **4,137** nodes (a **74%+** reduction).
3. Compact abstract state representations are critical to avoiding state-space explosion when formulating complex multi-goal search problems.